gems/activerecord-2.3.2/lib/active_record/attribute_methods.rb

```
module ActiveRecord
  module AttributeMethods
    def method_missing(method_id, *args, &block)
      method_name = method_id.to_s

      if self.class.private_method_defined?(method_name)
        raise NoMethodError.new("Attempt to call private method", method_name, args)
      end

      # If we haven't generated any methods yet, generate them, then
      # see if we've created the method we're looking for.
      if !self.class.generated_methods?
        self.class.define_attribute_methods
        if self.class.generated_methods.include?(method_name)
          return self.send(method_id, *args, &block)
        end
      end

      # ...
    end

    def read_attribute(attr_name)         # ...
    def write_attribute(attr_name, value) # ...
    def query_attribute(attr_name)        # ...
```

When you call a method such as Task#description= for the first time, the call is delivered to method_missing. Before it does its job, method_missing ensures that you're not inadvertently bypassing encapsulation and calling a private method. Then it calls an intriguing-sounding define_attribute_methods method.

We'll look at define_attribute_methods in a minute. For now, all you need to know is that it defines read, write, and query *Dynamic Methods (51)* for all the columns in the database. The next time you call description= or any other accessor that maps to a database column, your call isn't handled by method_missing. Instead, you call a real, non-ghost method.

When you entered method_missing, description= was a *Ghost Method (57)*. Now description= is a regular flesh-and-blood method, and method_missing can call it with a *Dynamic Dispatch (49)* and return the result. This process takes place only once for each class that inherits from ActiveRecord::Base. If you enter method_missing a second time for any reason, the class method generated_methods? returns true, and this code is skipped.

The following code shows how define_attribute_methods defines non-ghostly accessors.